Projet MLSF – Partie I
Un solveur SAT pour le jeu Meowdoku

Hugo Vigna Salomé Fonvielle

Modélisation logique et systèmes formels

Nous nous sommes limités au cas $N = M = K$: grille carrée, un chat par ligne et par colonne.

Question 1 : modéliser Meowdoku en CNF

Variables. Pour une grille $n \times n$, la variable $x_{i,j}$ vaut vrai s'il y a un chat en (i, j). Elle est numérotée $i \cdot n + j + 1$ (avec i, j à partir de 0), donc de 1 à n^2.

Clauses. Il y en a deux sortes :

— *conflits* : pour chaque paire de cases a, b qui ne peuvent pas porter un chat toutes les deux, la clause $(\neg x_a \vee \neg x_b)$. Deux cases sont en conflit si elles sont sur la même ligne, la même colonne, la même région, ou collées (8-voisinage, diagonales comprises) ;

— *au moins un chat par région* : pour chaque région, la clause $(x_a \vee x_b \vee \dots)$ sur toutes ses cases.

Correction. Les clauses de conflit donnent au plus un chat par ligne, par colonne et par région. Avec n régions et au moins un chat dans chacune, il y a exactement un chat par région. Les n chats sont donc sur n lignes différentes, soit exactement un par ligne, et de même par colonne. L'encodage teste toutes les paires de cases (`meowdoku.py`, fonction `encode`), soit $n^2(n^2 - 1)/2$ paires : il prend un temps en $O(n^4)$. Mais chaque case n'est en conflit qu'avec $O(n)$ autres (sa ligne, sa colonne, sa région, ses voisines), donc le nombre de clauses produites est en $O(n^3)$ en pratique (1 331 clauses pour $n = 10$, 11 755 pour $n = 20$) et en $O(n^4)$ seulement dans le pire cas d'une région couvrant presque toute la grille. La fonction `is_valid` contrôle une configuration directement avec les règles du jeu, sans passer par la CNF.

Générateur (`generator.py`). La méthode garantit l'existence d'une solution :

1. on place n chats, un par ligne et par colonne, jamais collés (choix aléatoire avec retour arrière) ;
2. chaque chat est la graine d'une région ; on fait grossir les régions case par case, en ajoutant au hasard une case libre voisine d'une région, jusqu'à remplir la grille.

Les chats initiaux forment donc une solution. Elle n'est pas forcément unique.

Question 2 : lecture DIMACS et DPLL

`dimacs.py` lit un fichier DIMACS : une formule est un ensemble (`set`) de `frozenset` d'entiers signés, donc les clauses en double disparaissent. `dpll.py` contient l'algorithme 1 (`dpll_sat`) et l'algorithme 2 (`dpll`, qui renvoie aussi la valuation). Chaque appel recrée une nouvelle formule avec `simplifier`.

Validation. Nous avons utilisé les 41 instances publiques DIMACS (`public-instances/`). Lors d'un lancement sans limite de temps, le statut obtenu avec `dpll` est identique à `expected.json` pour les 41 fichiers, et chaque modèle trouvé satisfait toutes les clauses.

Performances. Nous avons lancé `python sat.py instance.cnf --mode dpll --output-model` (choix aléatoire, 60 s maximum, une exécution par instance, `bench_q6.py`). Sur les 30 instances `public-*`, 22 sont résolues en moins d'une seconde et 26 en moins de 3 s ; `public-18` prend 3,2 s et `public-27` 41,5 s. `public-03` (UNSAT) et `public-06` (SAT) dépassent 60 s. Les temps varient beaucoup d'une exécution à l'autre à cause du hasard : lors d'un lancement précédent sans limite de temps, `public-06` avait été résolue en 0,16 s, `public-12` en 65 s (ici 1,1 s) et `public-03` en 117 s. Les instances UNSAT coûtent en général plus cher que les SAT, car il faut explorer tout l'arbre.

Question 3 : résoudre une grille

`solve_grid.py` lit la grille, la convertit en CNF, appelle DPLL, puis écrit le fichier au format de la figure 2 du sujet (dictionnaire « région : position », puis grille avec les symboles `*`). La solution est toujours vérifiée avec `is_valid`. Sur la grille du sujet (`data/grids/sujet.txt`), la sortie est identique à la figure 2.

Question 4 : heuristiques de branchement

Dans `heuristics.py`, nous avons implémenté H0, H1, H2 (statique et dynamique), H3d, H4d, H7 (statique et dynamique) et une heuristique personnelle, H8. H3d et H7 sont fondées sur un littéral, H4d et H7 utilisent la taille des clauses, et plusieurs sont dynamiques. Ce choix couvre les trois contraintes du sujet et permet de comparer, à score égal, une version statique et une version dynamique (H2 et H7). H0 et H1 servent de références sans réflexion. Nous n'avons pas fait H5 ni H6, proches de H4 et H7.

H8. Elle exploite la structure de l'encodage : les seules clauses entièrement positives sont les clauses « au moins un chat dans la région ». H8 prend la région qui a le moins de cases encore possibles, choisit dans cette région la variable la plus fréquente et la met à 1. Une case à 1 élimine immédiatement toutes les cases en conflit avec elle. S'il n'y a plus de clause positive, H8 se rabat sur H7d.

Vérification. Chaque heuristique donne le même statut que l'algorithme 2 sur les instances de la question 2, et les modèles sont valides.

Question 5 : comparaison des heuristiques

Nous avons rajouté des compteurs à l'algorithme DPLL (décisions, retours arrière, propagations unitaires, littéraux purs) et mesuré le temps CPU et la part passée dans l'heuristique (`bench_q5.py`). Les 9 variantes (H0, H1, H2s, H2d, H3d, H4d, H7s, H7d, H8) ont tourné sur 160 instances : 40 de 3-SAT aléatoire, 60 grilles Meowdoku à solution plantée (donc SAT), 60 grilles à régions aléatoires (27 SAT, 33 UNSAT). Timeout de 10 s, une exécution par cas, résultats bruts dans `results/q5.csv`.

Résultat final. Pour chaque famille : nombre d'instances résolues, temps CPU moyen par instance (en s, un timeout compte 20 s) et nombre moyen de décisions (sur les instances résolues seulement). Sur le 3-SAT le temps moyen est d'au plus 0,02 s, donc nous ne comparons que les

décisions. Pour les grilles UNSAT nous ne donnons pas les décisions : la moyenne ne porterait que sur les instances résolues, qui diffèrent d'une variante à l'autre.

	3-SAT	Meow plantée (60)			Meow aléat. SAT (27)			Meow aléat. UNSAT (33)	
	décis.	rés.	CPU	décis.	rés.	CPU	décis.	rés.	CPU
H0	115	60	0,25	849	26	0,94	778	26	4,51
H1	120	60	0,24	491	27	0,10	200	30	2,24
H2s	22	59	0,47	739	25	2,27	5537	25	5,10
H2d	18	60	0,06	204	25	1,52	191	26	4,62
H3d	24	60	0,06	204	25	1,52	191	26	4,61
H4d	12	60	0,06	169	25	1,51	130	27	4,26
H7s	26	59	0,47	739	25	2,25	5537	25	5,10
H7d	18	60	0,08	204	25	1,52	191	25	4,95
H8	17	60	0,02	9	27	0,03	17	33	0,11

Classement.

— **3-SAT aléatoire :** H4d (12 décisions), H8 (17), H2d = H7d (18), H2s (22), H3d (24), H7s (26), puis loin derrière H0 (115) et H1 (120). Les heuristiques dynamiques font 5 à 10 fois moins de décisions que le choix aléatoire.

— **Meowdoku :** H8 est première partout : 9 décisions en moyenne sur les grilles plantées contre 169 pour la meilleure des autres (H4d), et c'est la seule qui résout les 33 grilles UNSAT (H1 en résout 30, H4d 27, les autres 25 ou 26). Derrière H8, le classement dépend de la famille. Sur les grilles plantées : H2d, H3d, H4d et H7d (0,06 à 0,08 s), puis H0 et H1 (0,25 s), puis H2s et H7s (0,47 s, avec un timeout). Sur les grilles à régions aléatoires, H1 est deuxième, devant H0 et les dynamiques, et les statiques sont dernières.

Explications.

1. **Les heuristiques classiques (H2 à H7) passent à côté de la structure de Meowdoku.** Chaque variable est positive dans une seule clause et négative dans des dizaines. H2d, H3d, H4d et H7d choisissent donc la valeur 0 pour 100 % de leurs décisions, ce qui ne propage presque rien. Elles font tout de même mieux que le hasard sur l'ensemble des grilles plantées (169 à 204 décisions pour H4d, H2d, H3d, H7d contre 849 pour H0 et 491 pour H1, voir le tableau), mais restent très loin de H8 (9). Sur les petites grilles elles sont même moins bonnes : à $n = 10$, 88 décisions pour H2d, H3d et H7d contre 17 pour H0 et 14 pour H1 (`results/q5_tableaux.txt`).
2. **Sur 3-SAT c'est l'inverse :** choisir la variable la plus fréquente marche, et H4d, qui ne compte que dans les clauses les plus courtes, est la meilleure. Le bon choix d'heuristique dépend donc de la structure du problème.
3. **H8 gagne sur Meowdoku** parce qu'elle exploite l'encodage : elle prend la clause « au moins un chat dans la région » la plus réduite et met une case à 1, ce qui élimine d'un coup toutes les cases en conflit. Sa limite : elle dépend de l'encodage (sur 3-SAT elle marche, 17 décisions, mais ne bat pas H4d).
4. **Statique contre dynamique.** Le dynamique recalcule tout à chaque décision : l'heuristique prend en moyenne 45 à 54 % du temps CPU des variantes dynamiques sur les grilles plantées (contre 25 % pour les statiques). Mais cela vaut le coup : sur les grilles plantées, H2s fait 739 décisions en moyenne et H2d 204 (tableau).

Limite. Une seule exécution par cas.

Question 6 : solveur optimisé

Idée générale. Nous sommes partis de `dpll.py` avec l'heuristique H8, puisque la question 5 montre que c'est la meilleure sur Meowdoku. Le problème de `dpll`, c'est qu'à chaque fois qu'on affecte un littéral, il reparcourt toute la formule : `simplifier` pour la mettre à jour, `trouver_unitaire` pour chercher une clause unitaire, `trouver_pur` pour un littéral pur. En profilant `dpll` avec H8 sur les grilles Meowdoku générées ($n = 10, 12, 14$), `simplifier` prend environ 40 % du temps, `trouver_unitaire` environ 30 % et H8 environ 20 %. Dans `optimized.py`, nous gardons exactement la même recherche (propagation, puis décision, branche « vrai » puis « faux », retour arrière) mais nous évitons ces parcours complets :

- `simplifier`**.** Au lieu de parcourir la formule à chaque affectation pour la mettre à jour (supprimer les clauses satisfaites et les littéraux faux), on ne la modifie jamais. On garde juste un tableau d'affectation `val` (pour chaque littéral : vrai, faux ou libre) et la liste `trail` des littéraux affectés dans l'ordre. Pour revenir en arrière, on dépile `trail` et on remet les cases de `val` à « libre ».
- `trouver_unitaire`**.** Pour chaque clause, on surveille deux de ses littéraux (`watch`). Tant que ces deux littéraux ne sont pas faux, la clause ne peut être ni unitaire ni fausse. Donc quand on affecte un littéral, on regarde seulement les clauses qui le faisaient intervenir, pas toute la formule. Pour chacune, si l'autre littéral surveillé est vrai, la clause est satisfaite et on passe. Sinon on cherche un autre littéral non faux à surveiller à la place. S'il n'y en a pas, la clause est unitaire si l'autre littéral surveillé est libre (on l'affecte), et en conflit s'il est faux.
- **H8.** On met de côté les clauses entièrement positives une seule fois au début, pour ne pas avoir à reparcourir toute la formule à chaque décision pour les retrouver. Une clause entièrement positive le reste toujours, donc cette liste ne change pas. Quand il n'y a plus de clause positive, H8 prend la première variable libre d'une liste triée une fois par nombre d'occurrences (H2 statique).

Résultats sur Meowdoku. Nous avons lancé `python sat.py grille.cnf --mode dpll|optimized --output-model` sur les 120 grilles Meowdoku générées pour la question 5 (10 par taille, plantées et à régions aléatoires), avec 60 s maximum (`bench_q6.py`). Le modèle est vérifié clause par clause. Les deux modes donnent le même statut partout où `dpll` termine. Le temps est le temps réel du processus, démarrage de Python compris (environ 0,01 s par exécution, d'où des totaux d'environ 0,2 s pour 10 petites grilles). Pour chaque taille, *total* est la somme des 10 grilles et *max* le temps de la plus longue ; un timeout (TO) compte 60 s. `dpll` est ici celui de la question 2, avec choix aléatoire, donc ses temps varient d'un lancement à l'autre.

Grilles	n	`dpll` : total	max	`optimized` : total	max
plantée	5	0,23 s	0,06 s	0,19 s	0,06 s
plantée	6	0,19 s	0,02 s	0,14 s	0,01 s
plantée	8	0,22 s	0,03 s	0,15 s	0,02 s
plantée	10	0,28 s	0,03 s	0,21 s	0,03 s
plantée	12	2,30 s	1,76 s	0,28 s	0,04 s
plantée	14	1,09 s	0,38 s	0,39 s	0,08 s
aléatoire	5	0,23 s	0,04 s	0,22 s	0,05 s
aléatoire	6	0,22 s	0,03 s	0,18 s	0,02 s
aléatoire	8	0,22 s	0,03 s	0,17 s	0,02 s
aléatoire	10	1,58 s	0,57 s	0,20 s	0,03 s
aléatoire	12	29,49 s	14,83 s	0,41 s	0,12 s
aléatoire	14	355,49 s (4 TO)	60,00 s	1,44 s	0,61 s

`optimized` résout les 120 grilles, la plus longue en 0,61 s. `dpll` dépasse 60 s sur 4 grilles à régions aléatoires de $n = 14$. Cet écart mélange deux effets : l'heuristique (H8 au lieu du hasard) et les structures de données. Pour isoler le second, nous avons comparé en Python `dpll.py` avec H8 à `optimized.py` sur les mêmes grilles (une seule exécution de chacun, car les deux sont déterministes) : sur les 10 grilles aléatoires de $n = 14$, 1,91 s contre 0,77 s, et sur celles de $n = 12$, 0,26 s contre 0,13 s. Donc les structures de données donnent un facteur 2 à 2,5.

Résultats sur les instances publiques. Même commande, sur les 41 fichiers de `public-instances/` (60 s maximum, une exécution, `results/public.csv`). Les deux modes donnent un résultat correct (statut conforme à `expected.json`, modèle valide) sur toutes les instances terminées : `dpll` en termine 39 sur 41, `optimized` 38. Le tableau ne garde que les instances qui prennent plus d'une demi-seconde avec l'un des deux ; toutes les autres prennent moins de 0,5 s avec les deux.

Instance	attendu	`dpll` (Q2, choix aléatoire)	`optimized`
`public-03`	UNSAT	>60 s	>60 s
`public-04`	SAT	0,02 s	0,71 s
`public-06`	SAT	>60 s	>60 s
`public-11`	SAT	0,97 s	0,13 s
`public-12`	SAT	1,13 s	>60 s
`public-15`	UNSAT	1,49 s	0,09 s
`public-18`	SAT	3,18 s	0,67 s
`public-19`	SAT	1,91 s	0,09 s
`public-22`	UNSAT	0,73 s	0,06 s
`public-27`	SAT	41,50 s	0,06 s
`public-30`	UNSAT	1,04 s	0,06 s

`optimized` est nettement meilleur sur la plupart des instances moyennes (`public-11`, `15`, `18`, `19`, `22`, `27`, `30`). Mais trois instances ne sont pas terminées en 60 s (`public-03`, `06`, `12`). `dpll` en termine une de plus (`public-12`) sur ce lancement, probablement grâce à un tirage aléatoire favorable (il avait pris 65 s une autre fois). Sur `public-04`, `optimized` est plus lent (0,71 s contre 0,02 s). Au total, sur les instances terminées, `optimized` cumule 3,1 s contre 54 s pour `dpll`.

Usage des assistants IA

Nous avons utilisé Claude (Claude Code) pour écrire une partie du code, les tests et des brouillons d'analyse. Pour chaque morceau, nous avons relu, fait expliquer et simplifié le code jusqu'à pouvoir le défendre. Les résultats chiffrés viennent de nos propres exécutions (`results/`), que l'on peut relancer avec `bench_q5.py` et `bench_q6.py`.